

Plateforme Mastering Live Debix

Spécification Phase 1a — V4.2

Console numérique 8 canaux end-to-end
Intègre investigations V4 (854s) + V4.1 (790s)

Document de référence technique

2026-04-22 — rev V4.2

Table des matières

1	Ce qui change depuis V4.1	2
1.1	Micro-fixes V4.2	2
1.2	Ce qui est confirmé unanimement par les 6 workers V4.1	2
1.3	Bugs V4 corrigés en V4.1	3
1.4	Ce qui est confirmé et inchangé	3
2	Architecture cible (identique V3/V4)	3
3	Phasage V4.1	4
4	Phase 1a.1 MVP — squelette m4 final V4.1	5
4.1	Architecture	5
4.2	sof-imx8mp-tac5212-masterV42.m4 (top-level)	5
4.3	sof-imx8mp-tac5212-pipe-effects-capture.m4	6
4.4	sof-imx8mp-tac5212-pipe-effects-playback.m4	7
4.5	Board config Kconfig (inchangé V4)	9
5	Test T5 bit-perfect — stratégie T5b recommandée	9
6	Tests Phase 1a.1 MVP V4.2	9
7	Phase 1a.2 — stratégie tap HP_Monitor	10
8	Livrables Phase 1a.1 MVP V4.2	10
9	Protocole	10

1 Ce qui change depuis V4.1

V4.1 a été soumis à l'investigation finale (6 workers : gemini 114s, kimi 363s, glm 438s, minimax 633s, claude-code 643s, qwen 785s). **Verdict unanime : GO pour Phase 1a.1 MVP** — le squelette passe la gate `m4 | alsatplg`. V4.2 intègre les micro-fixes recommandés et clarifie la stratégie T5b.

1.1 Micro-fixes V4.2

Fix	Correction V4.2
F1	<code>CONFIG_DRC_MAX_PRE_DELAY_FRAMES = 512</code> (et non 256 comme dit V4.1 §5). Pre-delay par défaut = <code>DRC_DEFAULT_PRE_DELAY_FRAMES = 256</code> qui correspond à <code>CONFIG_DRC_MAX_PRE_DELAY_FRAMES » 1</code> .
F2	Ajout 12e arg <code>SCHEDULE_TIME_DOMAIN_DMA</code> à <code>PIPELINE_PCM_ADD</code> pour hygiène et cohérence avec <code>DAI_ADD</code> (claude-code, pattern masterlite).
F3	Stratégie T5b formalisée : topology variante <code>masterV42-T5.m4</code> qui OMET le widget <code>W_DRC</code> de la chaîne pour le null test bit-perfect strict. C'est la seule voie sample-exact car <code>drc_s32_default</code> applique toujours le pre-delay même si <code>enabled=0</code> .
F4	Note CMakeLists : <code>GLOB sof/*.m4</code> (ligne 7 de <code>CMakeLists.txt</code>) capture automatiquement les fichiers pipeline, mais le top-level <code>masterV42.m4</code> doit être ajouté explicitement à <code>TPLGS</code> pour build standalone. La recipe Yocto <code>meta-local</code> peut aussi le compiler via invocation <code>m4 alsatplg</code> directe (pattern baseline).
F5	Note <code>DAI_ADD</code> : 12 args acceptés (pattern <code>DAI_ADD_SCHED</code> qui passe 12 args). Les 3 args manquants (<code>DAI_CHANNELS</code> , <code>DAI_RATE</code> , <code>DYNAMIC_PIPE</code>) sont inutilisés par <code>pipe-dai-capture.m4/pipe-dai-playback.m4</code> . Pas de modification nécessaire.

1.2 Ce qui est confirmé unanimement par les 6 workers V4.1

- **Gate compile** : le squelette pass `m4 | alsatplg`
- `multiband_drc_coef_passthrough.m4` existe dans notre fork (70 lignes, 2 bands, `enable_emp_deemp=0`)
- `W_PIPELINE` créé UNIQUEMENT par `pipe-dai-{capture,playback}.m4` via `DAI_ADD` — un seul scheduler widget par `pipeline_id`
- Pattern kcontrol "`PIPELINE_ID Master Capture Volume`" **correct** : `m4` expand `PIPELINE_ID` dans les strings double-quotées
- `W_MULTIBAND_DRC` / `W_DRC` en (2, 2) confirmé par upstream
- `W_PGA` capture (2, `DAI_PERIODS`), playback (`DAI_PERIODS`, 2)
- Exports `PIPELINE_SCHED_COMP_<id>` (capture) et `PIPELINE_PLAYBACK_SCHED_COMP_<id>` (playback) disponibles pour piggyback Phase 1a.2

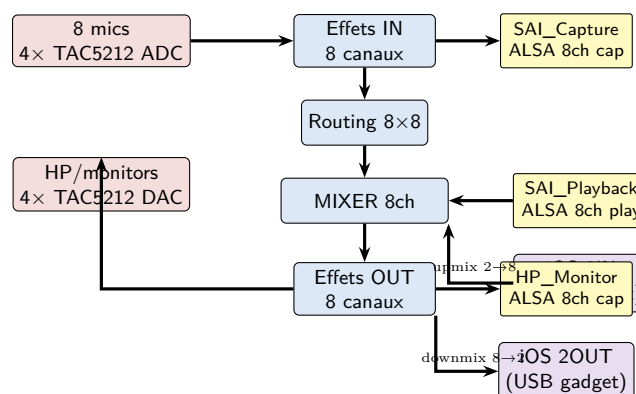
1.3 Bugs V4 corrigés en V4.1

#	Sévérité	Correction V4.1
B1	CRITICAL	SOF_DRC_MAX_BLOB_SIZE / SOF_MULTIBAND_DRC_MAX_BLOB_SIZE sont des <code>#define C</code> invisibles en m4. Remplacés par 1024 littéral (pattern upstream <code>pipe-drc-enabled-capture.m4:31</code>).
B2	CRITICAL	W_PIPELINE(N_PCMC/N_PCMP, ...) ajouté en V4 est redondant et buggué . Le scheduler widget est créé par <code>pipe-dai-capture.m4</code> via DAI_ADD sur le même pipeline_id. Ajouter un W_PIPELINE côté PCM crée un 2e scheduler → comportement indéfini. Supprimé en V4.1.
B3	CRITICAL	multiband_drc_coef_default.m4 est en mode ACTIF (emp_deemp=1). Remplacé par multiband_drc_coef_passthrough.m4 qui existe upstream et est un vrai bypass.
B4	HIGH	drc_coef_default.m4 est aussi en mode actif (enabled=1) et aucun blob passthrough DRC n'existe upstream . Pire : même avec enabled=0, drc_s32_default applique un pre-delay de 256 frames (~5.3 ms). Conséquence : T5 sample-perfect impossible avec DRC dans la chaîne . V4.1 adopte 3 stratégies (voir §5).
B5	HIGH	kcontrol PGA "DEF_PGA_VOLUME" ne matche pas le nom SectionControlMixer généré par C_CONTROLMIXER = "PIPELINE_ID Master Capture Volume". Corrigé.
B6	MEDIUM	CONTROLMIXER channel map : VOLUME_CHANNEL_MAP upstream (FL/FR) suffit — en mode linked, le PGA applique un gain unique aux 8 canaux. Pas besoin de 8 KCONTROL_CHANNEL explicites.
B7	MEDIUM	W_PGA periods : capture = (2, DAI_PERIODS, ...), playback = (DAI_PERIODS, 2, ...) — cohérence upstream <code>pipe-volume-capture.m4:47</code> et <code>pipe-volume-playback.m4</code> .

1.4 Ce qui est confirmé et inchangé

- Architecture console 8ch end-to-end, les 4 PCMs, phasage
- Tous SCHEDULE_TIME_DOMAIN_DMA via DAI_ADD (pas de TIMER)
- Pas de cross-pipeline en Phase 1a.1 MVP
- Pattern W_VENDORTUPLES / W_DATA pour DEF_PGA_CONF validé par gemini-3-pro
- CONFIG_COMP_MIXER=y pour Phase 1a.3 (mixer limité 2 sources)
- Root cause V2.7 -110 : PIPELINE_DEMUX_3 fantôme + guard is_same_sched SCHEDULE_TIME_DOMAIN mismatch. Pas "W_PIPELINE manquant" (erreur V3).

2 Architecture cible (identique V3/V4)

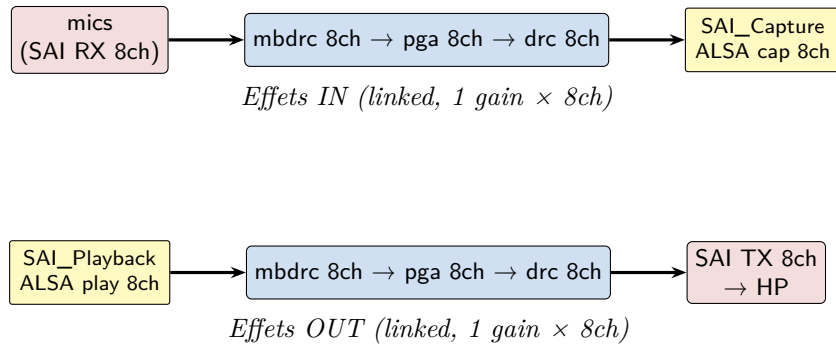


3 Phasage V4.1

Phase	Livrables	Objectif
1a.1 MVP	2 pipelines 8ch autonomes, effets linked (mbdrc+pga+drc), pas de cross-pipeline	Prouver pass-through + effets, trigger stable sans -110
1a.2	Tap HP_Monitor via option co-scheduled cross-pipeline same sched_comp	Capture post-effets OUT pour NPU
1a.3	Routing 8×8 + mixer 2 sources (mics routés + Playback) + injection SAI_Playback	iOS absorbé via routing slots 0-1, pas via mixer
1a.4	USB gadget iOS 2IN/2OUT	iOS connectable
2	USB ASIO gadget 8IN/8OUT	DAW sur host externe
3	Passage linked → per-channel	8 instances effets

4 Phase 1a.1 MVP — squelette m4 final V4.1

4.1 Architecture



4.2 sof-imx8mp-tac5212-masterV42.m4 (top-level)

```
#
# Topology V4.2 Phase 1a.1 MVP : console 8ch end-to-end, linked effects.
# 2 pipelines 8ch autonomes, SCHEDULE_TIME_DOMAIN_DMA partout (12e arg).
# Le scheduler widget W_PIPELINE est créé par pipe-dai-{capture,playback}.m4.
#
include('utils.m4')
include('dai.m4')
include('pipeline.m4')
include('sai.m4')
include('pcm.m4')
include('buffer.m4')
include('common/tlv.m4')
include('sof/tokens.m4')
include('platform/imx/imx8.m4')

# PIPE 1 : SAI7 RX 8ch -> mbdrc -> pga -> drc -> SAI_Capture host
# 12e arg SCHEDULE_TIME_DOMAIN_DMA ajouté V4.2 (hygiène, cohérence DAI_ADD)
PIPELINE_PCM_ADD(sof/sof-imx8mp-tac5212-pipe-effects-capture.m4,
    1, 0, 8, s32le,
    1000, 0, 0,
    48000, 48000, 48000,
    SCHEDULE_TIME_DOMAIN_DMA)

# PIPE 6 : SAI_Playback host -> mbdrc -> pga -> drc -> SAI7 TX 8ch
PIPELINE_PCM_ADD(sof/sof-imx8mp-tac5212-pipe-effects-playback.m4,
    6, 1, 8, s32le,
    1000, 0, 0,
    48000, 48000, 48000,
    SCHEDULE_TIME_DOMAIN_DMA)

# DAI_ADD crée aussi le W_PIPELINE pour pipeline_id via pipe-dai-*.m4
DAI_ADD(sof/pipe-dai-capture.m4,
    1, SAI, 7, tac5212-hifi,
    PIPELINE_SINK_1, 2, s32le,
    1000, 0, 0, SCHEDULE_TIME_DOMAIN_DMA)

DAI_ADD(sof/pipe-dai-playback.m4,
    6, SAI, 7, tac5212-hifi,
    PIPELINE_SOURCE_6, 2, s32le,
    1000, 0, 0, SCHEDULE_TIME_DOMAIN_DMA)

PCM_CAPTURE_ADD(SAI_Capture, 0, PIPELINE_PCM_1)
PCM_PLAYBACK_ADD(SAI_Playback, 1, PIPELINE_PCM_6)

DAI_CONFIG(SAI, 7, 0, tac5212-hifi,
    SAI_CONFIG(DSP_A, SAI_CLOCK(mclk, 12288000, codec_mclk_in),
        SAI_CLOCK(bclk, 12288000, codec_consumer),
        SAI_CLOCK(fsyc, 48000, codec_consumer),
        SAI_TDM(8, 32, 255, 255),
        SAI_CONFIG_DATA(SAI, 7, 0)))
```

4.3 sof-imx8mp-tac5212-pipe-effects-capture.m4

```
# Capture 8ch : DAI SAI RX -> B3 -> MBDRC -> B2 -> PGA -> B1 -> DRC -> B0 -> HOST
# Pattern direct : pipe-volume-capture.m4 + pipe-drc-enabled-capture.m4 + mbdrc.

include('utils.m4')
include('buffer.m4')
include('pcm.m4')
include('dai.m4')
include('bytecontrol.m4')
include('mixercontrol.m4')
include('pipeline.m4')
include('multiband_drc.m4')
include('pga.m4')
include('drc.m4')

# PGA volume kcontrol (nom complet = "<PIPELINE_ID> Master Capture Volume")
C_CONTROLMIXER(Master Capture Volume, PIPELINE_ID,
    CONTROLMIXER_OPS(volsw,
        256 binds the mixer control to volume get/put handlers, 256, 256),
    CONTROLMIXER_MAX(, 32),
    false,
    CONTROLMIXER_TLV(TLV 32 steps from -64dB to 0dB for 2dB, vtlv_m64s2),
    Channel register and shift for Front Left/Right,
    VOLUME_CHANNEL_MAP)

# PGA vendor tuples (pattern pipe-volume-capture.m4)
define(DEF_PGA_TOKENS, concat('pga_tokens_', PIPELINE_ID))
define(DEF_PGA_CONF, concat('pga_conf_', PIPELINE_ID))
W_VENDORTUPLES(DEF_PGA_TOKENS, sof_volume_tokens,
LIST('      ', 'SOF_TKN_VOLUME_RAMP_STEP_TYPE  "0"'
      , 'SOF_TKN_VOLUME_RAMP_STEP_MS    "250"'))
W_DATA(DEF_PGA_CONF, DEF_PGA_TOKENS)

# MBDRC byte control + blob BYPASS (multiband_drc_coef_passthrough.m4 existe upstream)
define(MULTIBAND_DRC_priv, concat('mbdrc_bytes_', PIPELINE_ID))
define(MY_MULTIBAND_DRC_CTRL, concat('mbdrc_control_', PIPELINE_ID))
include('multiband_drc_coef_passthrough.m4')
C_CONTROLBYTES(MY_MULTIBAND_DRC_CTRL, PIPELINE_ID,
    CONTROLBYTES_OPS(bytes,
        258 binds the control to bytes get/put handlers, 258, 258),
    CONTROLBYTES_EXTOPS(
        258 binds the control to bytes get/put handlers, 258, 258),
    , , ,
    CONTROLBYTES_MAX(, 1024),
    ,
    MULTIBAND_DRC_priv)

# DRC byte control + blob (default ACTIF : pour T5 bit-perfect, voir section T5)
define(DRC_priv, concat('drc_bytes_', PIPELINE_ID))
define(MY_DRC_CTRL, concat('drc_control_', PIPELINE_ID))
include('drc_coef_default.m4')
C_CONTROLBYTES(MY_DRC_CTRL, PIPELINE_ID,
    CONTROLBYTES_OPS(bytes,
        258 binds the control to bytes get/put handlers, 258, 258),
    CONTROLBYTES_EXTOPS(
        258 binds the control to bytes get/put handlers, 258, 258),
    , , ,
    CONTROLBYTES_MAX(, 1024),
    ,
    DRC_priv)

# Host PCM endpoint
W_PCM_CAPTURE(PCM_ID, SAI Capture, 0, 2, SCHEDULE_CORE)

# Widgets : mbdrc -> pga -> drc
W_MULTIBAND_DRC(0, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE,
    LIST('      ', "MY_MULTIBAND_DRC_CTRL"))
W_PGA(0, PIPELINE_FORMAT, 2, DAI_PERIODS, DEF_PGA_CONF, SCHEDULE_CORE,
    LIST('      ', "PIPELINE_ID Master Capture Volume"))
W_DRC(0, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE,
    LIST('      ', "MY_DRC_CTRL"))

# Buffers : B0 host side, B3 DAI side
W_BUFFER(0, COMP_BUFFER_SIZE(2,
    COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
```

```

    COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
    PLATFORM_HOST_MEM_CAP)
W_BUFFER(1, COMP_BUFFER_SIZE(2,
    COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
    COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
    PLATFORM_COMP_MEM_CAP)
W_BUFFER(2, COMP_BUFFER_SIZE(2,
    COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
    COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
    PLATFORM_COMP_MEM_CAP)
W_BUFFER(3, COMP_BUFFER_SIZE(DAI_PERIODS,
    COMP_SAMPLE_SIZE(DAI_FORMAT), PIPELINE_CHANNELS,
    COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
    PLATFORM_DAI_MEM_CAP)

# PAS DE W_PIPELINE ici (bug V4 B2).
# Scheduler widget fourni par pipe-dai-capture.m4 lors de DAI_ADD.

# Graph DAPM : capture = flux DAI -> HOST, dapm(sink, source)
P_GRAPH(pipe-effects-capture, PIPELINE_ID,
    LIST(
        'dapm(N_PCMC(PCM_ID), N_BUFFER(0))',
        'dapm(N_BUFFER(0), N_DRC(0))',
        'dapm(N_DRC(0), N_BUFFER(1))',
        'dapm(N_BUFFER(1), N_PGA(0))',
        'dapm(N_PGA(0), N_BUFFER(2))',
        'dapm(N_BUFFER(2), N_MULTIBAND_DRC(0))',
        'dapm(N_MULTIBAND_DRC(0), N_BUFFER(3))'))

# Exports pour DAI_ADD
indir('define', concat('PIPELINE_SINK_', PIPELINE_ID), N_BUFFER(3))
indir('define', concat('PIPELINE_PCM_', PIPELINE_ID), SAI Capture PCM_ID)

PCM_CAPABILITIES(SAI Capture PCM_ID, CAPABILITY_FORMAT_NAME(PIPELINE_FORMAT),
    PCM_MIN_RATE, PCM_MAX_RATE, 2, PIPELINE_CHANNELS, 2, 16,
    192, 16384, 65536, 65536)

undefine('MY_MULTIBAND_DRC_CTRL') undefine('MULTIBAND_DRC_priv')
undefine('MY_DRC_CTRL') undefine('DRC_priv')
undefine('DEF_PGA_TOKENS') undefine('DEF_PGA_CONF')

```

4.4 sof-imx8mp-tac5212-pipe-effects-playback.m4

```

# Playback 8ch : HOST -> B0 -> MBDRD -> B1 -> PGA -> B2 -> DRC -> B3 -> DAI SAI TX
# Pattern direct : pipe-multiband-drc-playback.m4 + pga + drc chainage.

include('utils.m4')
include('buffer.m4')
include('pcm.m4')
include('dai.m4')
include('bytecontrol.m4')
include('mixercontrol.m4')
include('pipeline.m4')
include('multiband_drc.m4')
include('pga.m4')
include('drc.m4')

C_CONTROLMIXER(Master Playback Volume, PIPELINE_ID,
    CONTROLMIXER_OPS(volsw,
        256 binds the mixer control to volume get/put handlers, 256, 256),
    CONTROLMIXER_MAX(, 32),
    false,
    CONTROLMIXER_TLV(TLV 32 steps from -64dB to 0dB for 2dB, vtlv_m64s2),
    Channel register and shift for Front Left/Right,
    VOLUME_CHANNEL_MAP)

define(DEF_PGA_TOKENS, concat('pga_tokens_', PIPELINE_ID))
define(DEF_PGA_CONF, concat('pga_conf_', PIPELINE_ID))
W_VENDORTUPLES(DEF_PGA_TOKENS, sof_volume_tokens,
    LIST(
        'SOF_TKN_VOLUME_RAMP_STEP_TYPE "2"',
        'SOF_TKN_VOLUME_RAMP_STEP_MS "20"'))
W_DATA(DEF_PGA_CONF, DEF_PGA_TOKENS)

# MBDRD bypass (blob passthrough upstream)

```

```

define(MULTIBAND_DRC_priv, concat('mbdrc_bytes_', PIPELINE_ID))
define(MY_MULTIBAND_DRC_CTRL, concat('mbdrc_control_', PIPELINE_ID))
include('multiband_drc_coef_passthrough.m4')
C_CONTROLBYTES(MY_MULTIBAND_DRC_CTRL, PIPELINE_ID,
CONTROLBYTES_OPS(bytes,
258 binds the control to bytes get/put handlers, 258, 258),
CONTROLBYTES_EXTOPS(
258 binds the control to bytes get/put handlers, 258, 258),
, , , CONTROLBYTES_MAX(, 1024), ,
MULTIBAND_DRC_priv)

# DRC default blob (actif, voir section T5)
define(DRC_priv, concat('drc_bytes_', PIPELINE_ID))
define(MY_DRC_CTRL, concat('drc_control_', PIPELINE_ID))
include('drc_coef_default.m4')
C_CONTROLBYTES(MY_DRC_CTRL, PIPELINE_ID,
CONTROLBYTES_OPS(bytes,
258 binds the control to bytes get/put handlers, 258, 258),
CONTROLBYTES_EXTOPS(
258 binds the control to bytes get/put handlers, 258, 258),
, , , CONTROLBYTES_MAX(, 1024), ,
DRC_priv)

# Host playback endpoint
W_PCM_PLAYBACK(PCM_ID, SAI Playback, 2, 0, SCHEDULE_CORE)

# Widgets : mbdrc -> pga (W_PGA periods inversés pour playback) -> drc
W_MULTIBAND_DRC(0, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE,
LIST(' ', "MY_MULTIBAND_DRC_CTRL"))
W_PGA(0, PIPELINE_FORMAT, DAI_PERIODS, 2, DEF_PGA_CONF, SCHEDULE_CORE,
LIST(' ', "PIPELINE_ID Master Playback Volume"))
W_DRC(0, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE,
LIST(' ', "MY_DRC_CTRL"))

# Buffers : B0 host side, B3 DAI side
W_BUFFER(0, COMP_BUFFER_SIZE(2,
COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
PLATFORM_HOST_MEM_CAP)
W_BUFFER(1, COMP_BUFFER_SIZE(2,
COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
PLATFORM_COMP_MEM_CAP)
W_BUFFER(2, COMP_BUFFER_SIZE(2,
COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
PLATFORM_COMP_MEM_CAP)
W_BUFFER(3, COMP_BUFFER_SIZE(DAI_PERIODS,
COMP_SAMPLE_SIZE(DAI_FORMAT), PIPELINE_CHANNELS,
COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
PLATFORM_DAI_MEM_CAP)

# PAS DE W_PIPELINE ici (scheduler fourni par pipe-dai-playback.m4)

# Graph DAPM : playback = flux HOST -> DAI, dapm(sink, source)
P_GRAPH(pipe-effects-playback, PIPELINE_ID,
LIST(' ',
'dapm(N_BUFFER(0), N_PCMP(PCM_ID))',
'dapm(N_MULTIBAND_DRC(0), N_BUFFER(0))',
'dapm(N_BUFFER(1), N_MULTIBAND_DRC(0))',
'dapm(N_PGA(0), N_BUFFER(1))',
'dapm(N_BUFFER(2), N_PGA(0))',
'dapm(N_DRC(0), N_BUFFER(2))',
'dapm(N_BUFFER(3), N_DRC(0))'))

# Exports pour DAI_ADD
indir('define', concat('PIPELINE_SOURCE_', PIPELINE_ID), N_BUFFER(3))
indir('define', concat('PIPELINE_PCM_', PIPELINE_ID), SAI Playback PCM_ID)

PCM_CAPABILITIES(SAI Playback PCM_ID, CAPABILITY_FORMAT_NAME(PIPELINE_FORMAT),
PCM_MIN_RATE, PCM_MAX_RATE, 2, PIPELINE_CHANNELS, 2, 16,
192, 16384, 65536, 65536)

undefine('MY_MULTIBAND_DRC_CTRL') undefine('MULTIBAND_DRC_priv')
undefine('MY_DRC_CTRL') undefine('DRC_priv')
undefine('DEF_PGA_TOKENS') undefine('DEF_PGA_CONF')

```


4.5 Board config Kconfig (inchangé V4)

```
CONFIG_IMX8M=y
CONFIG_HAVE_AGENT=n
CONFIG_FORMAT_CONVERT_HIFI3=n
CONFIG_KPB_FORCE_COPY_TYPE_NORMAL=n

# Phase 1a.1 MVP
CONFIG_COMP_DRC=y
CONFIG_COMP_VOLUME=y
CONFIG_COMP_CROSSOVER=y
CONFIG_COMP_MULTIBAND_DRC=y
CONFIG_COMP_MUX=y

# Ajouté V4 pour Phase 1a.3
CONFIG_COMP_MIXER=y
```

5 Test T5 bit-perfect — stratégie T5b recommandée

Précision V4.2 : `drc.h:26` définit `CONFIG_DRC_MAX_PRE_DELAY_FRAMES = 512` et `drc.h:29` définit `DRC_DEFAULT_PRE_DELAY_FRAMES = CONFIG_DRC_MAX_PRE_DELAY_FRAMES » 1 = 256` frames. Le pre-delay effectif par défaut = 256 frames (~5.3 ms @48k) mais le max configurable est 512.

Même avec `enabled=0` côté DRC, la fonction `drc_s32_default` (`drc_generic.c:744-750`) appelle `drc_delay_input_sample_s32` qui applique le pre-delay. La fonction zero-latency `drc_default_pass = audio_stream_copy` (`drc_generic.c:473`) n'est sélectionnée **que si** `cd->config == NULL` (`drc.c:362`). Dès qu'un blob est chargé, le pre-delay est inévitable.

Trois stratégies T5 :

1. **T5a compensé** : loopback HW, null test avec compensation de latence (~256 frames DRC + buffers). Tolérance ± 1 LSB. Faisable, pragmatique.
2. **T5b sans DRC (RECOMMANDÉ V4.2)** : topology variante `sof-imx8mp-tac5212-masterV42-T5.m4` qui omet `W_DRC` et le buffer associé. Graph réduit : `DAI RX → B2 → MBDR → B1 → PGA → B0 → HOST`. Null test sample-perfect atteignable (MBDR en passthrough blob + PGA bit-perfect à `VOL_ZERO_DB=0x10000`). **Seule voie sample-exact** confirmée par les 6 workers.
3. **T5c blob patché** : créer localement un `drc_coef_zerolatency.m4` avec `enabled=0` ET `pre_delay_time=0`. Mais `drc_set_pre_delay_time()` clampe à `DRC_DIVISION_FRAMES=32` min → delay minimum 32 frames garanti. Non sample-exact. Abandonné.

Choix V4.2 : **T5b** pour validation bit-perfect stricte de la gate. La topology MVP principale (`masterV42.m4`) reste avec DRC actif pour les tests T1-T4 (trigger, full-duplex, contrôles).

6 Tests Phase 1a.1 MVP V4.2

1. **T0 Compile** : `m4 -I. -I m4 -I sof -I common -I platform/common common/abi.m4 masterV42.m4 | alsatplg -c - -o masterV42.tplg` sans erreur, aucun widget orphelin. **Gate obligatoire** avant deploy.
2. **T0a Widget count** : `alsatplg -dump masterV42.tplg | grep -c "type: scheduler"` doit retourner **2** (preuve : 1 scheduler par pipeline, créé par `pipe-dai-*.m4`). Total ~22 widgets.
3. **T1 Load** : kernel charge sans -22 ni -110. Carte `softac5212tdm` + 2 PCMs visibles.
4. **T2 Playback** : `aplay -D plughw:softac5212tdm,1 -c 8 -f S32_LE tone.wav`.
5. **T3 Capture** : `arecord -D plughw:softac5212tdm,0 -c 8 -f S32_LE -d 3 mics.wav`.
6. **T4 Full-duplex** : T2 + T3 simultanés 60 s, pas de xrun.
7. **T5 Bit-perfect** : stratégie **T5b** avec topology variante `masterV42-T5.m4` (sans `W_DRC`). Loopback HW TAC OUT → TAC IN, null test via `sox -m -v 1 in.wav -v -1 rec.wav null` → attendu ≤ 1 LSB.

8. **T6 Contrôles** : `amixer controls | grep -E "Master|MBDRC|DRC"` liste "1 Master Capture Volume", "6 Master Playback Volume", 4 byte controls MBDRC/DRC. `amixer cset name='1 Master Capture Volume' 32` change le gain sans glitch.
9. **T7 Charge CPU** : lecture via `sof-logger -t /etc/sof/sof-logger.toml -f /lib/firmware/imx/sof/so` ou `debugfs /sys/kernel/debug/sof/etrace` des cycles kcps ou cpc. Attendu < 30%.

7 Phase 1a.2 — stratégie tap HP_Monitor

L'investigation V4 a confirmé : **aucun exemple upstream** de `W_PCM_CAPTURE` comme sink d'un widget dans une pipeline `PLAYBACK`. Le framework `IPC3` ne l'interdit pas explicitement (`host_legacy.c` détermine direction par widget type, pas par position dans la chaîne), mais le pattern est non testé.

Stratégie V4.2 retenue : **option co-scheduled cross-pipeline same sched_comp** (pattern `sof-imx8mp-tac5212-masterlite.m4:49-56` confirmé upstream avec `PIPELINE_SCHED_COMP_3`).

- Pipeline playback (PIPE 6) enrichie : `W_MUXDEMUX` inséré en fin de chaîne effets alimente 2 sinks : B3 (vers DAI TX) et B4 (vers pipeline capture `HP_Monitor`). Export `PIPELINE_DEMUX_6 = N_MUXDEMUX(0)`.
- Pipeline `HP_Monitor` (PIPE 7) : pipeline capture avec 13e arg `PIPELINE_PLAYBACK_SCHED_COMP_6` (exporté par `pipe-dai-playback.m4:24`). Le `sched_comp = N_DAI_OUT(6)` est partagé → `guard is_same_sched` (`pipeline-stream.c:505`) passe.
- DAPM cross-pipeline : `SectionGraph` top-level avec `dapm(PIPELINE_SINK_7, PIPELINE_DEMUX_6)`. Le widget `PIPELINE_DEMUX_6` doit être **explicitement exporté** par le fichier `playback` (bug V2.7 à ne pas reproduire : `masterlite.m4:88` référençait `PIPELINE_DEMUX_3` non-exporté).

Fallback : loopback HW (câble TAC OUT → TAC IN) si le pattern cross-pipeline tap reste instable.

8 Livrables Phase 1a.1 MVP V4.2

- `sof/tools/topology/topology1/sof-imx8mp-tac5212-masterV42.m4` (top-level)
- `sof/tools/topology/topology1/sof/sof-imx8mp-tac5212-pipe-effects-capture.m4`
- `sof/tools/topology/topology1/sof/sof-imx8mp-tac5212-pipe-effects-playback.m4`
- `sof/tools/topology/topology1/sof-imx8mp-tac5212-masterV42-T5.m4` (variante T5 sans `W_DRC`)
- Patch `sof/app/boards/imx8mp-evk-mimx8ml8-adsp.conf` : `+CONFIG_COMP_MIXER=y`
- Entrée `sof/tools/topology/topology1/CMakeLists.txt` `TPLGS` : ajouter `"sof-imx8mp-tac5212-masterV42"` (ou compile via recipe Yocto meta-local directement)
- `meta-local/recipes-kernel/linux/files/test-phase1a1-mvp.sh` (tests T0–T7)
- Suppression des fichiers V2.7 buggés : `masterlite.m4`, `pipe-masterchain.m4`, `pipe-mastertap.m4`, `pipe-sai-merge.m4`, `mux_route_sai_merge.m4`, `demux_route_default.m4`

9 Protocole

- Aucune modification source SOF sans `critic_analyze` préalable (mode DOUX, fenêtre 2000 s post-analyze).
- Un changement = un commit = validation utilisateur.
- Gate m4 | `alsatplg` OBLIGATOIRE avant tout deploy sur board.
- Si T0 échoue, retour à `critic_analyze` avec l'erreur exacte **avant** de modifier le squelette.
- Backup kernel+firmware baseline avant premier deploy (rescue).